KapasAI — Agricultural AI Platform

Zarkhez (MultiAgent System)

Evaluation Report

Powered by LangGraph | Traced via LangSmith

Test Date: June 4, 2026 | Model: GPT-4o-mini

Project: KapasAI | Workspace: Personal

4	0%	6.02s	10.1K
Total Runs	Error Rate	P50 Latency	Total Tokens

1. Executive Summary

This report presents the results of the first formal evaluation of the KapasAI MultiAgent System **Zarkhez** an AI-powered assistant built specifically for cotton farmers in Pakistan. The system was tested on June 4, 2026 using the LangSmith tracing platform, which automatically recorded every conversation, measured response speed, and counted costs.

The evaluation ran **4 conversation turns** across a single session, covering three different types of user queries: a greeting, a memory check, and an agricultural disease question. All 4 runs completed successfully with **zero errors**, demonstrating that the system is stable and ready for further testing.

4 **Total Runs** All completed successfully	**0%** **Error Rate** No failures detected	**6.02s** **P50 Latency** Median response time
11.20s **P99 Latency** Slowest 1% of responses	**10.1K** **Total Tokens** Across all 4 runs	**$0.001** **Total Cost** Estimated API spend

2. What Is Zarkhez?

Zarkhez is an agricultural AI assistant designed to help cotton farmers diagnose crop diseases, get prevention advice, and answer farming questions — in simple, everyday language. It is built using a **multi-agent architecture**, which means it is not one single AI brain but a team of specialized AI agents that work together:

Supervisor Agent	The 'manager' of the system. It reads every user message and decides which specialist should answer. For example, if a farmer asks about a disease, it routes the question to the Disease Expert.
Disease Expert Agent	A specialist that searches through a knowledge base of cotton diseases. It uses RAG (Retrieval-Augmented Generation) — a technique where the AI looks up real documents before answering, rather than guessing.
Spray and Weather expert agent	It Provides accurate weather updates, forecasts, and smart spraying recommendations based on current climate conditions.

Framework: LangGraph (for agent orchestration) | **Model:** GPT-4o-mini by OpenAI | **Tracing:** LangSmith | **Knowledge Base:** Local cotton disease documents

3. Detailed Test Run Results

Below is a breakdown of each conversation turn that was tested. Think of each 'run' as one exchange between the user and the AI system.

Run 1	**Status: PASS**
User Said:	Hi, I am Mubashir.
AI Responded:	Hello Mubashir! How can I assist you today?
Agent Used:	General Conversation
Response Time:	2.02s
Tokens Used:	253
API Cost:	$0.00004335
Analysis:	Simple greeting handled correctly. The agent responded politely and retained the user's name for future turns.

Run 2	Status: PASS
User Said:	What is my name?
AI Responded:	Your name is Mubashir. How can I help you further?
Agent Used:	General Conversation
Response Time:	1.14s
Tokens Used:	280
API Cost:	$0.00004830
Analysis:	Memory retention test. The system correctly recalled 'Mubashir' from the previous turn — proving that the conversation history is being passed properly between messages. This was also the fastest response at just 1.14 seconds.

Run 3	Status: PASS
User Said:	My cotton leaves are turning yellow, what disease is this?
AI Responded:	Detailed diagnosis covering Leaf Variegation, Nutrient Deficiencies, Leaf Reddening, and Cotton Mosaic Virus (CMV).
Agent Used:	Disease Expert (via Supervisor routing)
Response Time:	11.24s
Tokens Used:	2,776
API Cost:	$0.00052260
Analysis:	This was the most complex test. The Supervisor correctly identified this as an agricultural question and routed it to the Disease Expert. The expert then used the RAG tool to search the knowledge base and returned a comprehensive, well-structured answer with multiple possible causes.

Run 4	Status: PASS
User Said:	What is the correct [pesticide/treatment]?
AI Responded:	FINISH (with full treatment recommendation)
Agent Used:	Spray and Weather expert agent
Response Time:	10.02s
Tokens Used:	6,803
API Cost:	$0.00097590
Analysis:	Follow-up question from Run 3. The agent correctly continued the conversation context and provided treatment advice. This was the most token-heavy run due to the detailed agricultural response.

4. Performance Summary

The table below gives a quick side-by-side comparison of all four runs, so you can spot patterns at a glance.

Run	User Query	Agent	Latency	Tokens	Cost	Result
Run 1	Hi, I am Mubashir.	General	2.02s	253	$0.000043	PASS
Run 2	What is my name?	General	1.14s	280	$0.000048	PASS
Run 3	Cotton leaves yellow?	Disease Expert	11.24s	2,776	$0.000523	PASS
Run 4	Correct treatment?	Spray Expert	10.02s	6,803	$0.000976	PASS

Token & Cost Breakdown

Run	Prompt Tokens	Completion Tokens	Total Tokens	Cost (USD)
Run 1	241	12	253	$0.000043
Run 2	507	26	280*	$0.000048
Run 3	1,581	236	2,776	$0.000523
Run 4	~5,900	~900	6,803	$0.000976
TOTAL	~8,229	~1,174	~10,112	$0.001590

* Run 2 cumulative token count includes full conversation history passed to the model.

5. Key Observations & Insights

■ Perfect Reliability

All 4 runs completed without a single error. The system handled three very different types of queries (greeting, memory recall, and disease diagnosis) flawlessly. For an early evaluation, this is an excellent baseline.

■ Smart Routing Works

The Supervisor Agent correctly identified which specialist to use for each question and it correctly routed the questions. Spray-related questions went to the Spray_weather_expert_agent, while agricultural questions went to the Disease Expert agent.

■ Memory Is Intact

When asked 'What is my name?' in Run 2, the system correctly recalled 'Mubashir' from Run 1. This confirms that conversation history is being passed through the system properly.

■ Knowledge Base Is Being Used

In Run 3, the Disease Expert successfully retrieved relevant documents about cotton leaf conditions before answering — not making up information, but grounding its answer in the actual knowledge base. This is a critical feature for an agricultural AI.

■■ Latency on Complex Queries

Simple questions took 1–2 seconds. Complex disease queries took 10–11 seconds. This is expected because the system needs to route the question, search the knowledge base, and generate a detailed response. As the system grows, latency optimization will be important for a good farmer experience.

■■ Token Usage Grows with Complexity

Run 4 used 6,803 tokens — the highest of all runs. This is because the full conversation history is included in every request. For long conversations, this will increase costs. A conversation summarization strategy should be considered for production use.

6. Strengths & Areas for Improvement

What Went Well	Areas to Improve
Zero Errors System ran without any failures across all test scenarios.	**Latency on RAG** Disease queries take 10-11s. Target should be under 5s for field use.
Correct Routing The Supervisor correctly directed every query to the right specialist.	**Token Accumulation** Costs will rise as conversations grow longer. Needs a memory compression strategy.
Memory Retention User context (name) was preserved across conversation turns.	**Test Coverage** Only 4 scenarios tested. Need broader coverage: pest queries, weather, market prices.
RAG Functioning Disease Expert retrieved relevant knowledge before answering.	**No Evaluation Scores** LangSmith feedback scores were not configured yet. Add LLM-as-judge scoring.
Low Cost Total spend was under $0.002 for 4 runs — very economical.	**Single Session Only** Tests were run in one session. Multi-session memory persistence needs testing.

7. Recommendations & Next Steps

1	**Expand Test Coverage** Add at least 20-30 diverse test cases covering pest identification, irrigation advice, market price queries. More varied tests give a much clearer picture of system capability.
2	**Add Automated Evaluation Scores** Configure LangSmith's evaluators to automatically score each response on correctness, relevance, and completeness. This removes the need for manual review and creates a repeatable benchmark.
3	**Optimize Response Latency** Investigate caching frequently-asked disease queries, or pre-loading common knowledge base sections. A target of under 5 seconds for 95% of queries would make the system feel responsive in the field.
4	**Implement Conversation Summarization** Instead of passing the full conversation history with every request, summarize older parts of the conversation. This will reduce token usage and costs significantly as conversations get longer.
5	**Test Multi-Session Memory** Verify that the system can remember a user across different conversation sessions (e.g., a farmer who asked about a disease yesterday can follow up today).
6	**Integrate into the Evaluation Folder** Store this report and future test runs in the /evaluation/ directory of the project. Build a versioned log so the team can track how the system improves over time.

8. Conclusion

The first evaluation of the **Zarkhez** has been a strong success. The system demonstrated **100% reliability**, correct agent routing, memory retention, and grounded agricultural responses using its knowledge base — all at a remarkably low cost of under $0.002.

This is an early prototype evaluation, but the results show the system is built on a solid foundation. The next phase should focus on broader test coverage, automated scoring, and latency improvements to bring the system closer to a production-ready state for real farmers in the field.

Overall Verdict: Ready for Phase 2 Testing.